



Brainbox AI for Odoo

How it works

How Odoo talks to your brainBox server, what every file in the add-on does, how the keys and roles keep each company's data separate, and how to install and configure it.

ODOO ADD-ON

brainbox_ai 17.0.2.0.0

WEB SDK

spres-web 2.0.0

DATE

8 October 2026

Written for the project owner, to read and to explain to management. Plain-language explanations come first; technical details follow in "For developers" boxes and in the reference sections.

Contents

1	Executive summary	The whole story on one page	3
2	The big picture	Odoo, websites and brainBox in two diagrams	4
3	A message's journey	"When is my bill due?" step by step	6
4	Companies (tenants) and keys	pk_, sk_ and the admin token explained	8
5	Who sees what	Roles, audiences and why the browser can't cheat	11
6	If another company installs the add-on	Does it talk to my brainBox? Onboarding checklist	13
7	Installation and configuration	Every setting, display modes, running Odoo	15
8	File-by-file reference	Every file of the add-on, routes, settings keys	19
9	The web SDK (sdk-web)	Plain-HTML snippet, options, Odoo mapping	25
10	Security and known limitations	What is protected, what is not (yet)	28
11	Glossary	The vocabulary in plain words	30
12	Where to find everything	Folders, files, menus and URLs	31

HOW TO READ THIS DOCUMENT

Sections 1 to 6 are written for everyone, including non-developers: read them to understand and explain the product. Sections 7 to 9 are the practical and technical reference for whoever installs and maintains it. Section 10 lists the protections and the current limitations honestly, so nobody is surprised.

ABOUT SECRETS IN THIS DOCUMENT

No real key, password or token appears here. Keys are written as placeholders such as `pk_live_XXX` and `sk_live_XXX`. The only real address used is the public brainBox server <https://port.smartpowerbilling.com>.

1 Executive summary

Brainbox AI is an AI assistant that answers questions from a company's own knowledge (documents, FAQs, past support tickets). It has three parts, and Odoo is only one of the places where it can appear.



brainBox = the brain

Runs on **your** server
(`port.smartpowerbilling.com`).
Keeps each client company's knowledge separate, finds the relevant passages, has the AI model write the answer, and remembers the conversations.



SDKs = the faces

Ready-made chat windows:
sdk-web (websites and Odoo),
sdk-react (React / Next.js) and
sdk-react-native (mobile apps). They show the chat; there is no AI inside them.



Odoo add-on = the bridge

brainbox_ai puts the web chat inside Odoo, adds a settings screen, and passes each message to brainBox with the company's secret key and the user's identity.

IN PLAIN WORDS

brainBox is a call centre with one expert per client company; the SDKs are the telephones; the Odoo add-on is the receptionist inside Odoo. She checks who is calling, dials the right expert with the company's private access code and passes the answer back. **She never answers questions herself and keeps no notes.**

Five facts to remember

Fact	What it means
Odoo contains no AI	All AI work happens on the brainBox server.
Odoo stores no chats	Conversations live on brainBox, per user; Odoo keeps only its settings.
The secret key never reaches browsers	It stays on the Odoo server; users' browsers only talk to Odoo.
Each company is sealed off	Each company is a separate "tenant"; a key opens only its own tenant.
You control access	Nobody can use your brainBox without a key you created, and you can revoke it.

What Odoo users see: a chat button in a corner of every Odoo screen (or a docked panel, or a sparkle icon in the top bar), an **AI Assistant** app with full-screen chat and past conversations, and optionally a chat box inside a specific form that tells the AI which record is open.

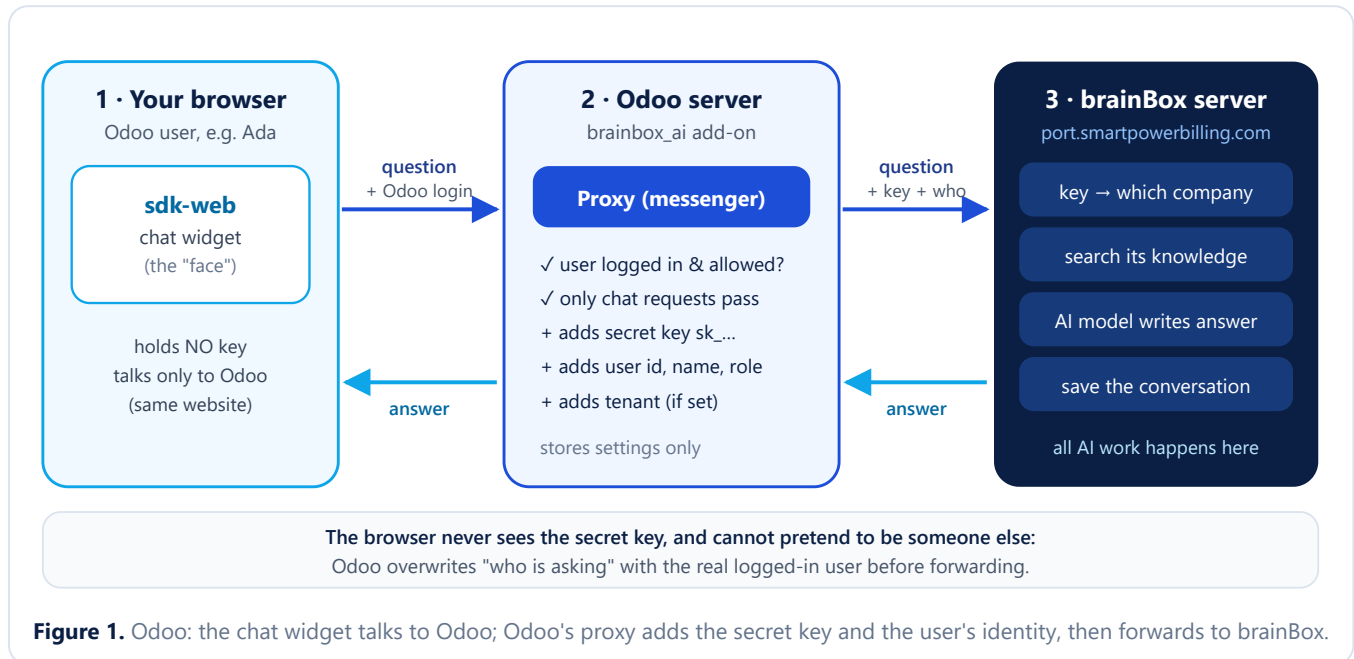
THE ONE-SENTENCE VERSION FOR YOUR BOSS

"Our AI runs on our own server and keeps every client company's knowledge in a sealed compartment; the Odoo add-on is just a secure doorway that shows our chat inside Odoo and passes questions to our server with that company's private key, so nobody can use it, or see another company's data, without a key we issued and can revoke at any time."

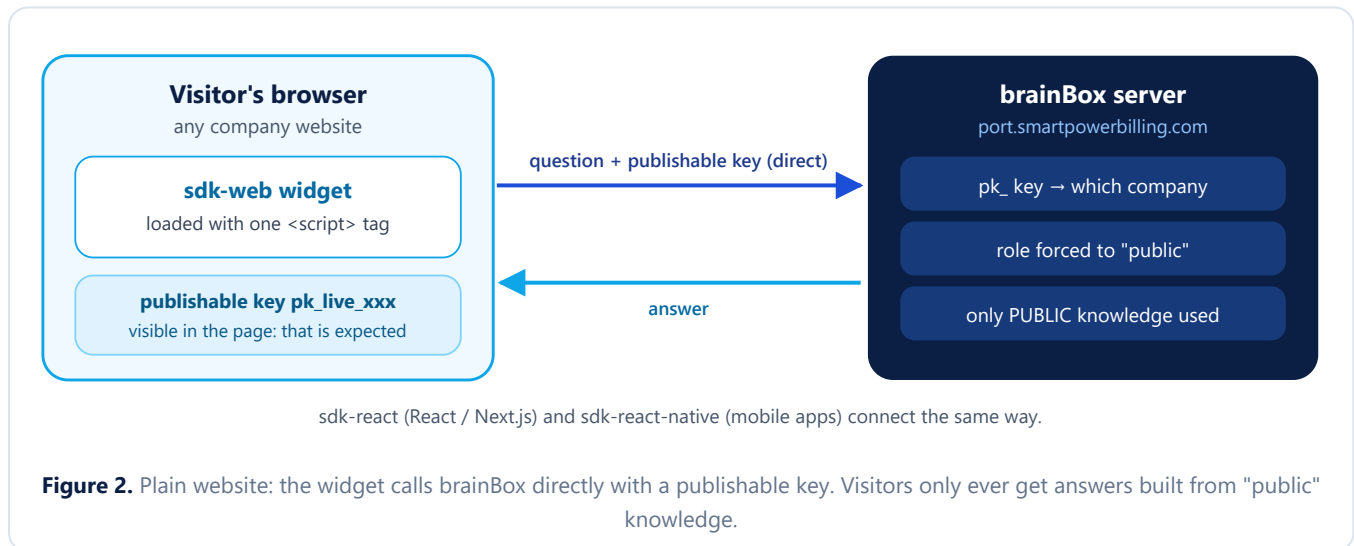
2 The big picture

The same brainBox server serves every channel. What changes between channels is **how the message reaches it** and **which key is used**.

A. Inside Odoo (the brainbox_ai add-on)



B. On a plain HTML website (no Odoo)



IN PLAIN WORDS

A website visitor is a stranger, so the website uses a **front-door key** that only lets people ask public questions. Odoo users are known, logged-in people, so Odoo can safely use the company's **office key** and tell brainBox who each user is.

FOR DEVELOPERS

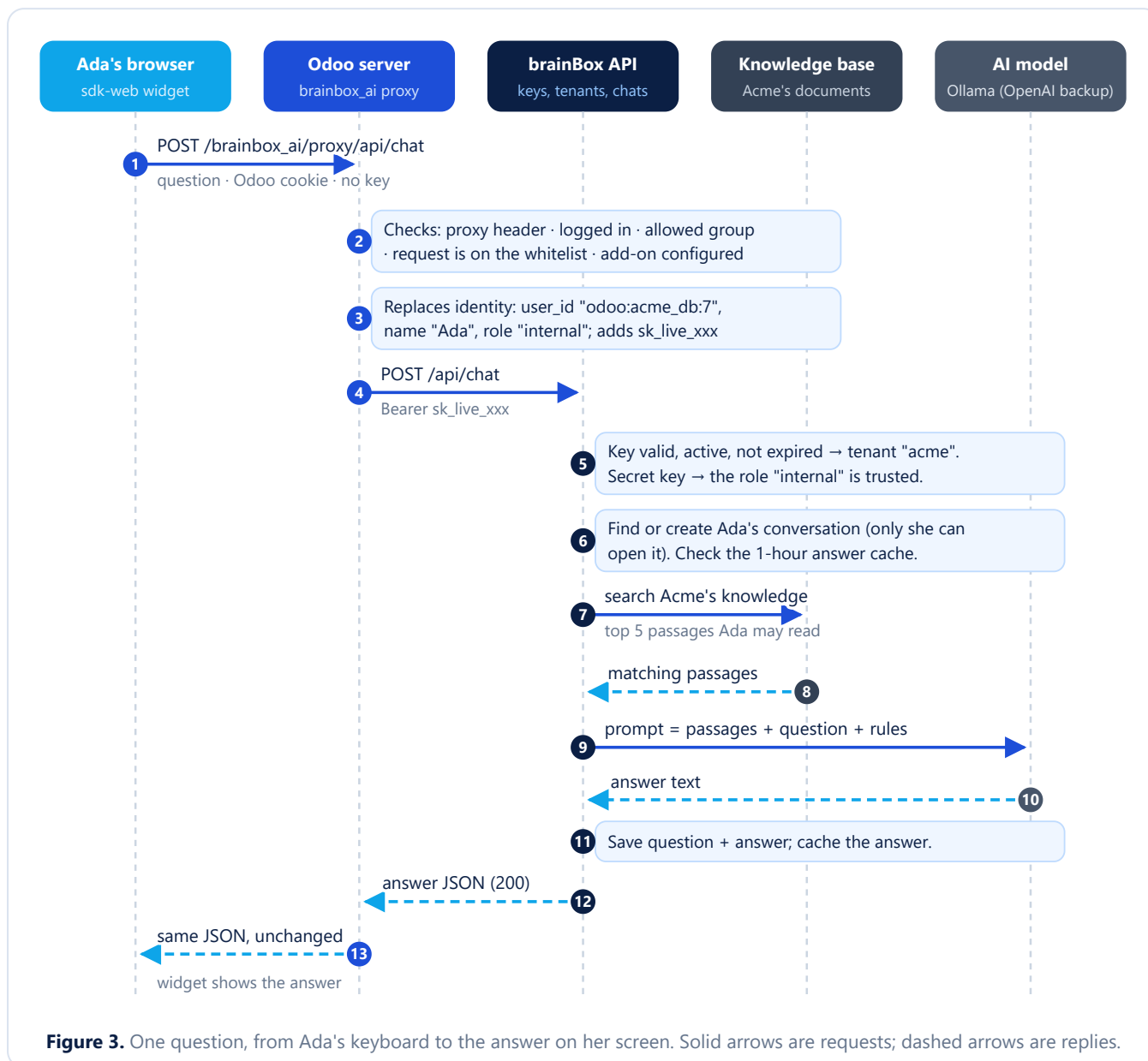
Odoo runs the SDK with `apiUrl`: `'/brainbox_ai/proxy'` and **no** `apiKey`. Because the proxy is on the same origin, there is no CORS and the Odoo session cookie authenticates the user. A website can also put its own proxy in front of the SDK ("proxy mode") to get the same protection.

The SDK family

SDK	Used for	Typical key
<code>sdk-web</code> (npm: <code>spres-web</code>)	Plain HTML, WordPress, any JavaScript site, and Odoo (bundled inside the add-on)	<code>pk_</code> in the page, or a server proxy holding <code>sk_</code> (as Odoo does)
<code>sdk-react</code>	React and Next.js web applications	<code>pk_</code> in the browser (keep any <code>sk_</code> on a server)
<code>sdk-react-native</code>	Mobile apps (iOS / Android)	<code>pk_</code> (anything inside an app can be extracted)

3 A message's journey

Example: **Ada** works in customer service at a company called **Acme**. She is logged into Acme's Odoo and types in the chat: "When is my bill due?"



The same steps in words

- 1 Ada asks.** She types the question and presses Enter. The chat window shows "Thinking...".
The SDK sends `POST /brainbox_ai/proxy/api/chat` to Odoo itself, with her Odoo login cookie and the header `X-Brainbox-Proxy: 1`. It contains no key and no tenant.

- 2 **Odoo checks the caller.** Is the request coming from the Odoo screen, is Ada logged in, is she in an allowed group, is this a permitted kind of request, is the add-on configured? If any answer is no, Odoo refuses and the chat shows a friendly error.

`controllers/proxy.py`: 403 without the header or for users outside "Allowed groups", 404 for paths not on the whitelist, 503 when not configured.

- 3 **Odoo writes the "envelope".** It removes anything the browser claimed about identity and writes the truth: Ada's id, her name, her role, and Acme's secret key.

Sets `user_id = "odoo:<database>:<user id>"`, `user_name`, `user_role` (from the role mapping), `tenant_id` (if set) and `Authorization: Bearer sk_live_xxx`.

- 4 **Odoo forwards to brainBox** at `https://port.smartpowerbilling.com/api/chat` and waits up to 200 seconds.

- 5 **brainBox identifies the company from the key.** The key is looked up (only its fingerprint is stored), must be active and not expired. It belongs to tenant "acme", so everything that follows happens inside Acme's compartment. Because it is a *secret* key, brainBox trusts the role "internal" sent by Odoo.

- 6 **brainBox opens Ada's conversation.** It is stamped with her user id, so no other user can open it. It also checks whether the same question was answered within the last hour for the same company and role (cache).

- 7 **brainBox searches Acme's knowledge** for the 5 most relevant passages, **but only among documents Ada's role may read** (for "internal": public, customer, vendor and internal documents).

- 8 **The passages come back** from the knowledge base (a PostgreSQL database with vector search).

- 9 **The AI model writes the answer** from those passages, with instructions such as "say you don't know rather than guess" and "never reveal other customers' details".

Ollama is called first; OpenAI is used as a fallback if configured.

- 10 **The answer comes back** from the model.

- 11 **brainBox saves** the question and the answer in Ada's conversation, and caches the answer for one hour.

- 12 **brainBox replies** to Odoo with the answer and the conversation id.

- 13 **Odoo passes the reply back unchanged**, and the widget displays it, formatted (lists, tables, links). The next time Ada opens the chat, her past conversations are listed.

IN PLAIN WORDS

If a **website visitor** asked the same question through a public website, steps 2 and 3 would not exist, the publishable key would make brainBox treat them as "public", and step 7 would search **only public documents**. Same brain, different access.

4 Companies (tenants) and keys

brainBox is shared by many companies. Two ideas keep them apart: **tenants** (separate compartments) and **keys** (who may enter which compartment, and with how much power).

Tenant: one company's separate brain

A **tenant** is one customer company on your brainBox server, identified by a short id such as `acme`. Everything brainBox stores is labelled with a tenant: the knowledge (documents, FAQs, tickets), the conversations, the cached answers, the keys. A search for Acme only looks at Acme's documents; a cached answer for Acme is never served to Globex.

Three kinds of keys

PUBLISHABLE · PK_LIVE_...

The front-door key
For public places: websites, mobile apps. Anyone may see it. It opens **one** company's front desk and lets visitors **ask questions as "public"**, nothing more.

SECRET · SK_LIVE_...

The office key
For servers only (Odoo, a company back-end). Opens **one** company's whole office: chat with trusted roles, and teach / manage the company's knowledge. Must never appear in a web page.

ADMIN TOKEN

The master key
Held only by you, the brainBox operator. It doesn't chat; it **creates, lists and revokes keys** for every company. Stored in the brainBox server's settings file.

What it can do	pk_publishable	sk_secret	Admin token
Ask questions (chat)	yes, as "public"	yes, any role	no
List / read conversations	yes	yes	no
Attach files / images to a chat	yes	yes	no
Assert a user role (internal, admin...)	no, ignored	yes	no
Teach the AI (train, ingest, delete sources)	no (403)	yes	no
Detailed health checks (db, cache, vector)	no	yes	no
Create / list / revoke keys	no	no	yes, all tenants
Scope	one tenant	one tenant	whole server
Where it is kept	web page / app code	Odoo Settings (server side) or another server's config	brainBox server <code>.env</code> (<code>BRAINBOX_ADMIN_TOKEN</code>)

The basic health check (`GET /api/health`) needs no key at all; it only says "the server is running".

IMPORTANT: THE TENANT COMES FROM THE KEY

A key belongs to exactly one tenant. If a request also names a tenant and it is a **different** one, brainBox refuses it (403 "tenant_id does not match API key"). In Odoo you may therefore leave **Tenant ID** empty when a secret key is set; if you fill it in, it must be exactly the key's tenant.

How keys are stored and created

- brainBox stores only a **fingerprint** (SHA-256 hash) of each key, plus its type, tenant, a name, a short visible prefix (e.g. `sk_live_Ab3x`) for recognising it, active/revoked status, optional expiry date and "last used" time.
- The full key is **shown only once**, at creation. If it is lost, create a new one and revoke the old one.
- An **existing** key can be imported (so already-deployed sites keep working); it must be at least 24 characters long.

Option 1: command line on the brainBox server (works without the admin token)

```
# create a secret key for Acme's Odoo server (prints the key once)
python -m app.cli keys create --tenant acme --type secret --name "Acme Odoo server"
# create a publishable key for Acme's public website, expiring 1 Jan 2027
python -m app.cli keys create --tenant acme --type publishable --name "Acme website" --expires
2027-01-01
# list keys (never prints full keys), optionally for one tenant
python -m app.cli keys list --tenant acme
# revoke key number 42 immediately
python -m app.cli keys revoke 42
# with Docker, run the same commands inside the API container
docker exec -i brainbox-api python -m app.cli keys list
```

Option 2: the admin API (requires the admin token)

```
POST https://port.smartpowerbilling.com/api/admin/keys { "tenant_id": "acme", "key_type":
"secret", "name": "Acme Odoo" }
GET https://port.smartpowerbilling.com/api/admin/keys?tenant_id=acme
DELETE https://port.smartpowerbilling.com/api/admin/keys/42
Header on every call: X-Admin-Token: <your admin token>
```

Without `BRAINBOX_ADMIN_TOKEN` set on the server, the admin API answers 503 (disabled); with a wrong token, 401. A staff dashboard for managing keys and knowledge is **coming** (being built now); this document covers the current mechanisms only.

What happens if a key leaks?

Key	What someone could do with it	What to do
pk_publishable	Nothing secret: ask questions as "public" (public knowledge only) and use your AI capacity. It can also list conversations that are not tied to a specific user. A pk_ is public by design, so "leaking" is expected.	If abused, revoke it (<code>keys revoke <id></code>), create a new one, update the website.
sk_secret	Serious. Read all of that company's knowledge (by claiming the "admin" role), read its users' conversations, add or delete its training data. Other companies are not affected.	Revoke immediately, create a new secret key, paste it into Odoo Settings (Secret API key), save.
Admin token	Critical. Create keys for any company, and so reach any company's data.	Change <code>BRAINBOX_ADMIN_TOKEN</code> in the server <code>.env</code> , restart brainBox, review <code>keys list</code> and revoke unknown keys.

FOR DEVELOPERS: ENFORCEMENT SWITCH

Keys are enforced when `REQUIRE_API_KEY=true` (the default). Setting it to `false` is a temporary migration mode in which no key is checked and the tenant and role are taken from the request; brainBox logs a warning at start-up. It should stay `true` in production. Keys are read from `X-API-Key` or `Authorization: Bearer` (`app/dependencies.py`).

5 Who sees what: roles and audiences

Inside one company, not everyone should see everything. brainBox uses two labels: every piece of knowledge has an **audience**, and every question has a **role**. A role can only read certain audiences.

Audience: who a document is for

Audience	Meant for
public	anyone, including anonymous website visitors
customer	logged-in customers
vendor	suppliers / vendors
internal	staff (default for new training sources)
admin	administrators only

Role: what a person may read

Role	Reads audiences
admin	everything
internal	public, customer, vendor, internal
customer (= portal)	public, customer
vendor	public, vendor
public	public only

How Odoo decides a user's role

The add-on looks at the user's Odoo groups and applies the **first** rule that matches (Settings › Brainbox AI › Roles):

#	Odoo user	Role sent	So they can read
1	Administrator (Settings rights)	admin	everything
2	Member of a "Vendor group"	vendor	public + vendor
3	Member of a "Customer group"	customer	public + customer
4	Other internal (employee) user	"Role of internal users": internal (default) or public	public, customer, vendor, internal (or public only)
5	Portal user	"Role of portal users": customer (default), portal, vendor or public	depends on the choice
6	Anyone else	public	public only

Vendor and customer groups are chosen by the Odoo administrator (any Odoo groups). Unknown or missing roles are always treated as **public** by brainBox: the safe default.

Why the browser cannot fake its role

1. **Odoo overwrites identity.** The proxy deletes any `tenant_id`, `user_id`, `user_name` or `user_role` sent by the browser and writes the real values from the Odoo login.

2. **Only secret keys may state a role.** With a publishable key, brainBox ignores the role and uses `public`, so a website visitor who edits the page code still gets public answers only.
3. **Filtering happens before the AI sees anything.** Documents the role may not read are never retrieved, so the AI cannot leak them, even by accident.
4. **Cached answers are kept per company and per role,** so a "public" user can never receive an answer that was built for "internal".
5. **Conversations belong to one user.** Each Odoo user's conversations are stamped with `odoo:<database>:<user id>`; another user asking for them gets "Session not found".

IMPORTANT: THE ADD-ON LIVES IN ODOO'S BACK-OFFICE

The widget is loaded in Odoo's back-office screens (the `web.assets_backend` bundle) and the AI Assistant menu is for internal users. Portal customers using the Odoo website/portal pages do not see it today, so the "portal" and "customer group" settings matter only for users you deliberately give back-office access. Who may use it at all is controlled by **Allowed groups** (empty = all internal users; administrators always).

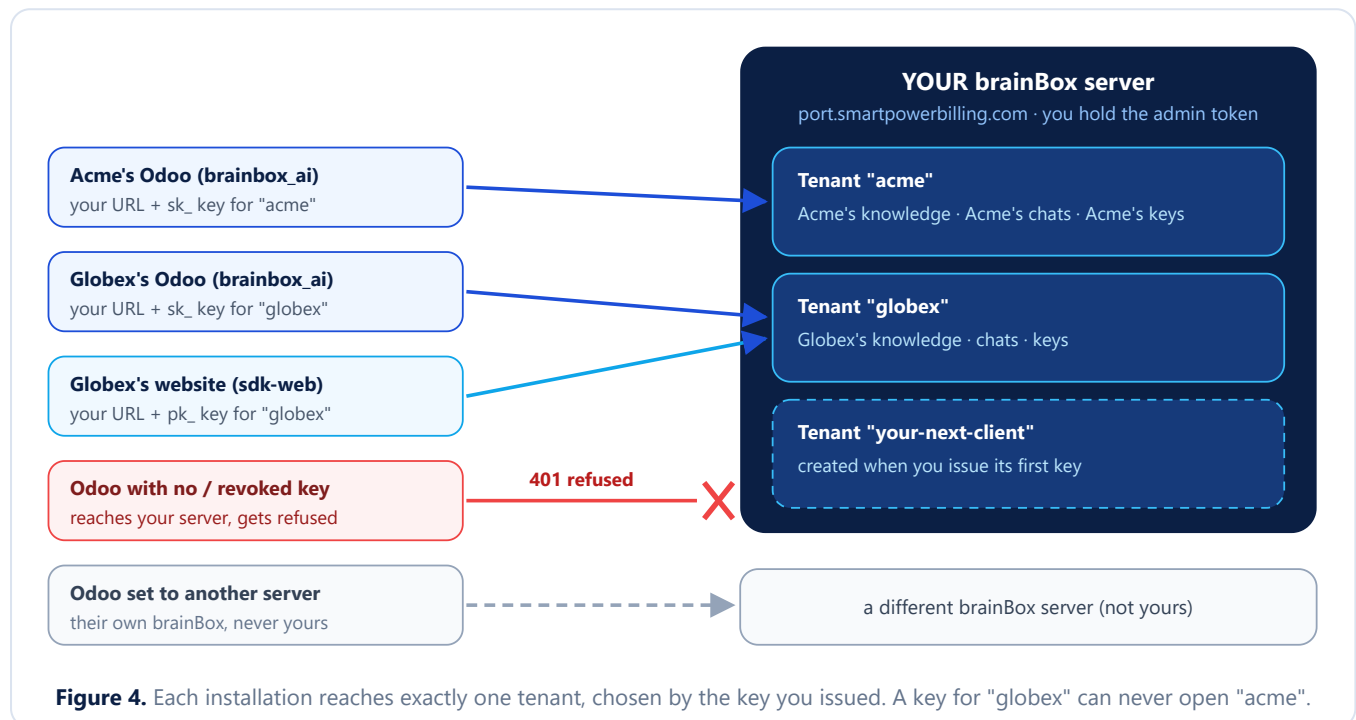
FOR DEVELOPERS

Role mapping: `brainbox.ai.backend._user_role()`. Audience filtering: `app/permissions.py` (`allowed_audiences`) applied in the search step `log_node`. Documents created before audiences existed are treated as `LEGACY_DOC_AUDIENCE` (default `public`). The audience of a training source can be changed later with `PATCH /api/train/sources/{id}`; this relabels all its documents and clears the company's cached answers.

6 If another company installs the add-on: does it talk to my brainBox?

SHORT ANSWER

Only if you let it. The add-on ships with **no server address and no key** (the Backend URL default is empty). A freshly installed copy talks to nobody. It reaches **your** brainBox only when the company's administrator enters **your server's URL** and **a key that you created for their tenant**. Without a valid key, your server refuses every chat request (401).



The rules, in plain words

- **The URL decides which server.** The Odoo administrator types the Backend URL. If they type yours, requests go to your server; if they run their own brainBox, they type that one instead and you are not involved.
- **The key decides which company.** On your server, the key alone decides the tenant. You create one tenant per customer company and give that company only its own keys.
- **Their data stays in their tenant.** Their knowledge, conversations and cached answers are labelled with their tenant and never mixed with other companies'.
- **You stay in control.** You can see every key (`keys list` : tenant, name, last used) and revoke any key at any time; the add-on then stops working for that company immediately, and their Odoo shows a "cannot get an answer" error banner instead of answers.
- **Several Odoo databases, one tenant.** A company with several Odoo databases can use the same tenant; users stay separate because their id includes the database name (`odoo:<database>:<user id>`).

Onboarding checklist for a new customer company

- ☐ **Choose the tenant id** (short, lowercase, e.g. `globex`).
- ☐ **Create a secret key** on your server: `python -m app.cli keys create --tenant globex --type secret --name "Globex Odoo"`. Copy the key shown once.
- ☐ **Send the key securely** (password manager or a one-time secret link, not plain email or chat) together with the Backend URL `https://port.smartpowerbilling.com`.
- ☐ **Load their knowledge** into their tenant with the training tools (secret key), and label each source with the right audience (public / customer / vendor / internal / admin).
- ☐ **They install the add-on** in Odoo (Apps > Update Apps List > Brainbox AI > Activate).
- ☐ **They configure Settings > Brainbox AI:** Backend URL, Secret API key; leave Tenant ID empty (or set exactly `globex`); choose display mode, allowed groups, role mapping and branding.
- ☐ **Test connection**, then ask a real question in the chat to confirm the key works (see the note below).
- ☐ **Optional website:** create a publishable key (`--type publishable`) for their public site and give them the sdk-web snippet (section 9).
- ☐ **Record it:** note the key ids, names and expiry dates so you can revoke or rotate them later.

NOTE ABOUT "TEST CONNECTION"

The Test connection button calls `/api/health`, which brainBox answers **without checking keys**. A green result proves the server is reachable, not that the key is right. A wrong key or tenant only shows up when the first real question is sent (the chat displays an error banner).

7 Installation and configuration

Install

- 1 Put the folder in place.** Copy `brainbox_ai` into an add-ons folder listed in `addons_path` of `odoo.conf` (here: `odoo\custom_addons`).
Depends on Odoo modules `base`, `web`, `base_setup` and the Python package `requests` (shipped with Odoo).
- 2 Install it.** In Odoo: **Apps > Update Apps List**, search "Brainbox AI", click **Activate**. Or from the command line (below).
- 3 Configure it.** **Settings > Brainbox AI** (also **AI Assistant > Configuration**). Only administrators see these settings.

Running Odoo (from `C:\Users\USER\Documents\PERSONAL\odoo`)

```
# start the server → http://localhost:8069
.venv\Scripts\python.exe odoo-bin -c odoo.conf
# install the add-on in database <db> (one-shot)
.venv\Scripts\python.exe odoo-bin -c odoo.conf -d <db> -i brainbox_ai --stop-after-init
# upgrade it after changing its files
.venv\Scripts\python.exe odoo-bin -c odoo.conf -d <db> -u brainbox_ai --stop-after-init
```

If a server already runs on port 8069, add `--http-port 8075` (or another free port) to the one-shot commands. After an upgrade, restart the running server (or open Odoo with `?debug=assets`) so browsers get the new files. Upgrading from version 1.x deletes the old `brainbox_ai_session` table (see the migration script); conversations are on brainBox and are not affected.

Every setting explained

Setting	What it does	Example / default
CONNECTION		
Backend URL	Address of the brainBox server, without <code>/api</code> . Odoo's server calls it; browsers never do.	<code>https://port.smartpowerbilling.com</code> default: empty
Tenant ID	The company's id on brainBox. Optional when a secret key is set (the key's tenant is used); if set, it must match the key's tenant exactly.	<code>acme</code> or empty
Secret API key (sk_...)	The company's office key. Saved on the Odoo server, never sent to browsers and never shown again (the form says "An API key is stored"). Leave empty to keep the current key; tick Remove it on save to delete it.	<code>sk_live_xxx</code>
SDK script URL	Optional: load the chat widget file from a CDN or your server instead of the copy bundled in the add-on. If that file cannot be loaded, the bundled copy is used.	<code>https://cdn.jsdelivr.net/npm/spres-web@2</code> default: empty (bundled)

Setting	What it does	Example / default
Test connection	Button: the Odoo server calls <code><Backend URL>/api/health</code> (10 s timeout) and shows success, warning or the error. Uses the values typed in the form.	"Connection successful"
DISPLAY		
Display mode	Where the chat appears in every Odoo app: floating, sidebar, systray or disabled (table below).	default: Floating
Position	Corner of the floating button and window.	Bottom right / Bottom left
Systray icon in floating mode	Also show the sparkle icon in the top bar when the floating button is used.	default: off
Allowed groups	Only users in one of these Odoo groups get the assistant (and may use the proxy). Empty = all internal users. Administrators always have access.	e.g. "Sales / User"
ROLES (SECTION 5)		
Role of internal users	Role sent for ordinary employees.	<code>internal</code> (default) or <code>public</code>
Role of portal users	Role sent for portal users.	<code>customer</code> (default), <code>portal</code> , <code>vendor</code> , <code>public</code>
Vendor groups	Users in these groups get the <code>vendor</code> role.	any Odoo groups
Customer groups	Users in these groups get the <code>customer</code> role.	any Odoo groups
BRANDING		
Colors	Primary color (buttons, user bubbles) and panel background. Must be hex colors.	<code>#b93fff</code> / <code>#fbf1ff</code>
Bot name	Name the assistant uses for itself.	Brainbox AI
Header title / subtitle	Text in the chat window header.	Brainbox AI / "Your AI assistant, everywhere in Odoo"
Launcher text	Label of the floating button.	Chat
Logo / Logo URL	Upload an image (stored in Odoo, served at <code>/brainbox_ai/logo</code>) or give an image address.	default: purple orb
Welcome message	One intro bubble per line. <code>{{name}}</code> = user's first name, <code>{{botName}}</code> = bot name.	"Hi {{name}}! I'm {{botName}}..."
Quick actions	One suggested question per line; clicking it sends it.	"What can you help me with?"

Display modes

Mode	What users see
Floating (default)	A chat button in a bottom corner of every Odoo app; it opens a chat window. A dock button in the chat header moves it into the sidebar and back.
Sidebar	A panel docked on the right under the Odoo top bar, opened and closed by a sparkle icon in the top bar. On wide screens (992 px and more) it pushes the page to the left so nothing is hidden. Users can pop it out to a floating window.
Systray	Only the sparkle icon in the top bar; it opens the floating chat window.
Disabled	Nothing global. The AI Assistant app and embedded widgets still work.

Each user's choice (floating or docked), whether the chat is open, and their last conversation are remembered **in their own browser** by the widget; nothing about it is stored in Odoo. If the add-on is not configured yet, administrators see the sparkle icon, and clicking it shows a notice with an **Open settings** button; other users see nothing.

The AI Assistant app

A new app in the Odoo home menu, **AI Assistant** (internal users), with two menus: **Chat** (a full-screen chat with the conversation list: New chat, search, conversations grouped by Today / Yesterday / This week / Older) and **Configuration** (administrators only, opens the settings). While this app is open, the floating button or sidebar is hidden and comes back when you leave.

Embedding the chat in a specific form

A developer can place the chat inside any Odoo view with one line, `<widget name="brainbox_chat" />`. By default the record being viewed is sent along with each question (e.g. "Odoo record: Contact — Acme Corporation (id 10)"), and each record keeps its own conversation. Example, adding it to the contact form, in your own module that depends on `brainbox_ai`:

```
<record id="view_partner_form_brainbox" model="ir.ui.view">
  <field name="name">res.partner.form.brainbox</field>
  <field name="model">res.partner</field>
  <field name="inherit_id" ref="base.view_partner_form" />
  <field name="arch" type="xml">
    <xpath expr="//sheet" position="inside">
      <widget name="brainbox_chat" height="520" title="Ask about this contact" />
    </xpath>
  </field>
</record>
```

Attribute	Default	Meaning
height	520	Height in pixels (minimum 240).
title	Header title from Settings	Title shown in the chat header.
record_context	1 (on)	☐ sends questions without the record description.

The view can also be created without code from Settings > Technical > Views in developer mode.

Updating the bundled widget (SDK)

The add-on contains an unmodified copy of the official web SDK. Never edit it inside Odoo: change the SDK project, then copy it in with the helper script, and upgrade the module (`-u brainbox_ai`) or restart Odoo.

```
# from the add-on folder: copy from the local SDK project
.\tools\sync_sdk.ps1 -SdkPath C:\Users\USER\Documents\PERSONAL\ai\sdk-web
# or download version 2.x of the npm package "spres-web" from the jsDelivr CDN
.\tools\sync_sdk.ps1 -Version 2
```

Alternative: keep the bundled copy and set **SDK script URL** to load the widget from a CDN or your server.

8 File-by-file reference

The add-on is deliberately thin: settings, a loader for the official widget, and a server-side messenger (proxy). No chat screens are built in Odoo and no chats are stored in Odoo.

```
brainbox_ai/
├── __manifest__.py      module card: name, version, what to load
├── __init__.py          loads controllers + models
├── README.md            developer documentation
├── controllers/
│   ├── __init__.py
│   ├── main.py          /brainbox_ai/config, /brainbox_ai/logo
│   └── proxy.py         /brainbox_ai/proxy/<path> (the messenger)
├── models/
│   ├── __init__.py
│   ├── brainbox_backend.py  helper: settings, who may use it, roles, health check
│   └── res_config_settings.py the Settings > Brainbox AI screen
├── migrations/17.0.2.0.0/post-migrate.py removes the 1.x chat table
├── static/
│   ├── description/icon.png  app icon
│   ├── lib/brainbox-web-sdk/brainbox-web-sdk.js  official SDK v2.0.0 (copy)
│   └── src/
│       ├── js/brainbox_service.js  glue: settings → Brainbox.init()
│       ├── js/brainbox_systray.js  sparkle icon in the top bar
│       ├── js/brainbox_chat_page.js AI Assistant app (full page)
│       ├── js/brainbox_chat_widget.js <widget name="brainbox_chat"/>
│       ├── xml/brainbox_ai.xml     small screen templates
│       └── scss/brainbox_ai.scss   styles for the Odoo-side parts
├── tools/sync_sdk.ps1         updates the SDK copy
└── views/
    ├── res_config_settings_views.xml  settings form + settings action
    └── brainbox_ai_menus.xml          AI Assistant app + menus
```

About "models" (database)

IN PLAIN WORDS

The add-on creates **no tables for chats or messages**. Its settings are saved as Odoo **system parameters** (one line per setting, all named `brainbox_ai....`).

Model	What it is
<code>brainbox.ai.backend</code>	AbstractModel : a helper with code only, no database table . Reads settings, decides who may use the assistant and with which role, builds the browser configuration, runs the health check.
<code>res.config.settings</code> (extended)	Odoo's standard settings screen, extended with the Brainbox AI fields. It is a temporary (transient) model: values are copied into system parameters on Save, and Odoo regularly cleans up the screen's temporary rows.

Model	What it is
Relation tables	The only tables the add-on causes Odoo to create are the automatic link tables of the three group pickers: <code>brainbox_ai_settings_group_rel</code> , <code>brainbox_ai_settings_vendor_group_rel</code> , <code>brainbox_ai_settings_customer_group_rel</code> (temporary rows of the settings screen; the real values are saved as comma-separated group ids). The new fields are also added as columns to Odoo's existing temporary <code>res_config_settings</code> table.

Settings saved as system parameters (`ir.config_parameter`)

Key	Holds	Key	Holds
<code>brainbox_ai.backend_url</code>	Backend URL	<code>brainbox_ai.primary_color</code>	Primary color
<code>brainbox_ai.tenant_id</code>	Tenant ID	<code>brainbox_ai.panel_color</code>	Panel color
<code>brainbox_ai.api_key</code>	Secret key (server only)	<code>brainbox_ai.bot_name</code>	Bot name
<code>brainbox_ai.sdk_script_url</code>	SDK script URL	<code>brainbox_ai.company_name</code>	Header title
<code>brainbox_ai.display_mode</code>	Display mode	<code>brainbox_ai.subtitle</code>	Header subtitle
<code>brainbox_ai.position</code>	Corner	<code>brainbox_ai.launcher_text</code>	Launcher text
<code>brainbox_ai.systray_in_floating</code>	Systray in floating	<code>brainbox_ai.logo</code>	Uploaded logo (base64)
<code>brainbox_ai.allowed_group_ids</code>	Allowed groups	<code>brainbox_ai.logo_url</code>	Logo URL
<code>brainbox_ai.vendor_group_ids</code>	Vendor groups	<code>brainbox_ai.welcome_message</code>	Welcome lines
<code>brainbox_ai.customer_group_ids</code>	Customer groups	<code>brainbox_ai.quick_actions</code>	Quick actions
<code>brainbox_ai.internal_role</code>	Role of internal users	<code>brainbox_ai.portal_role</code>	Role of portal users

Python files

`__manifest__.py`

module "ID card"

Tells Odoo what the module is and what to load.

- Name "Brainbox AI", version `17.0.2.0.0`, category Productivity, licence LGPL-3, `application: True` (appears as an app).
- Depends on `base`, `web`, `base_setup`; Python package `requests`.
- Data files: the settings view and the menus. Assets in `web.assets_backend`: the SDK file **first** (defines `window.Brainbox`), then the SCSS, JS and XML glue.

Standard Python loaders: import the controllers (`main` , `proxy`) and the models (`brainbox_backend` , `res_config_settings`). No logic.

controllers/main.py

non-secret settings & logo for the browser

Gives the browser what it needs to draw the chat, and nothing secret.

- `/brainbox_ai/config` (JSON-RPC, logged-in users): returns `get_public_config()` : allowed?, configured?, display mode, colours, names, logo URL, welcome messages, quick actions, position, SDK script URL. **Never the key.**
- `/brainbox_ai/logo` (GET, logged-in users): serves the uploaded logo, detecting PNG / JPEG / GIF / WebP / SVG, with a strict Content-Security-Policy and `nosniff` (an SVG cannot run scripts) and 7-day private caching.

controllers/proxy.py

the messenger between browser and brainBox

Receives the widget's requests on Odoo's own address and forwards the allowed ones to brainBox, adding the secret key and the real user identity. It contains no AI logic and returns brainBox's answer unchanged.

- Route `/brainbox_ai/proxy/<path>` , GET/POST, logged-in users only (`auth='user'`), `csrf=False` but requires the header `X-Brainbox-Proxy: 1` (a cross-site form cannot send a custom header, so this blocks cross-site request forgery).
- Refuses users outside Allowed groups (403), unknown paths (404), wrong methods (405), unconfigured module (503).
- Removes `tenant_id` , `user_id` , `user_name` , `user_role` from the browser's data (JSON body, query or upload form) and sets the true values; adds `Authorization: Bearer <secret key>` .
- Limits: JSON body 512 KB, uploads 10 MB, conversation ids checked against a safe pattern.
- Errors: 504 if brainBox is too slow, 502 if unreachable or if the answer is not JSON (friendly message, details only in the Odoo log).

Proxy whitelist: the only requests that pass

Browser calls (on Odoo)	Forwarded to brainBox	Methods	Content	Timeout
<code>/brainbox_ai/proxy/api/chat</code>	<code>/api/chat</code>	POST	JSON	200 s
<code>.../proxy/api/chat/sessions</code>	<code>/api/chat/sessions</code>	GET, POST	JSON	30 s
<code>.../proxy/api/chat/session</code>	<code>/api/chat/session</code>	POST	JSON	30 s
<code>.../proxy/api/chat/session/<id>/messages</code>	same path	GET	query	30 s
<code>.../proxy/api/chat/upload/file</code>	<code>/api/chat/upload/file</code>	POST	file ≤ 10 MB	60 s
<code>.../proxy/api/chat/upload/image</code>	<code>/api/chat/upload/image</code>	POST	image ≤ 10 MB	60 s
<code>.../proxy/api/health</code>	<code>/api/health</code>	GET	none	30 s

Everything else answers 404, in particular `api/train` , `api/ingest` and `api/admin` : teaching the AI and managing keys are impossible from an Odoo browser.

The add-on's "rule book": reads settings, decides access and roles, and checks the server.

- Constants: display modes, positions, role choices and **default values** (e.g. colour `#b93fff`, bot name "Brainbox AI", empty Backend URL).
- `_get_param`, `_backend_url`, `_api_key`, `_tenant`: read `brainbox.ai.*` system parameters.
`_is_configured`: Backend URL plus (Tenant ID or secret key).
- `_is_user_allowed`: public users never; administrators always; others if in Allowed groups (default: internal users).
- `_user_role`: the precedence admin > vendor groups > customer groups > internal > portal > public.
- `get_public_config`: the non-secret settings sent to the browser. `check_health`: GET `/api/health` with 10 s timeout and readable error messages. `BrainboxError`: errors safe to show to users.

Adds the Brainbox AI section to Odoo Settings and saves the values.

- Fields for every setting (section 7). Most are linked directly to a `brainbox.ai.*` system parameter.
- `get_values`: fills the form; the key is **never** sent back (only "a key is stored").
- `set_values`: only for administrators; saves or removes the key, welcome text, quick actions, group lists and the uploaded logo (validated as base64).
- Checks: SDK script URL must start with `https://`, `http://` or `/`; colours must be hex.
- `action_brainbox_test_connection`: runs the health check and shows a green / orange / red notification.

Version 1.x stored chats in Odoo; version 2 does not. This script deletes the leftover `brainbox_ai_session` table if it exists. (The old per-user display-mode column is removed by Odoo itself.)

Browser files (JavaScript, templates, styles)

The whole chat interface: button, window, sidebar, full page, history, uploads, formatted answers, error banners with Retry. It defines `window.Brainbox`. Identical to `ai\sdk-web\brainbox-web-sdk.js`; updated with `tools/sync_sdk.ps1`.

The glue between Odoo and the SDK.

- Fetches `/brainbox_ai/config` once, then converts the settings into `Brainbox.init()` options (table in section 9) with `apiUrl: '/brainbox_ai/proxy'`, header `X-Brainbox-Proxy: 1`, and no key or tenant.
- Loads the SDK: the bundled copy, or the "SDK script URL" with automatic fallback to the bundled copy.
- Creates the global chat (floating / sidebar / systray) after the Odoo web client is ready (4 s safety timer), measures the top bar for the sidebar and pushes `.o_action_manager` aside.
- Hides the global chat while the AI Assistant app is open. Shows administrators a "not configured" notice with an Open settings button.
- Small adapter: when old conversations are reloaded, hides the "Context: ... Question:" prefix added by embedded widgets so users only see their own question.
- Registered in the `services` registry as `brainbox_ai`.

Shows a sparkle icon that opens / closes the chat. Visible in sidebar and systray modes, in floating mode when "Systray icon in floating mode" is on, and for administrators when the add-on is not configured. Registered in the `systray` registry (`brainbox_ai.Systray`, sequence 30).

Client action `brainbox_ai.chat_page` (registry `actions`): runs the SDK in **page** mode inside Odoo (conversation list + chat), with its own memory key `bb-odoo:<db>:page`. Tells the service to hide the global chat while open.

Lets developers embed the chat in a form (SDK **inline** mode). Reads `height`, `title`, `record_context`. Adds a context line "Odoo record: <model name> — <record name> (id N)" to each question unless disabled, and keeps one conversation per record (`bb-odoo:<db>:rec:<model>:<id>`). Registered in the `view_widgets` registry.

Four small templates: `brainbox_ai.Systray` (the icon button), `brainbox_ai.Notice` ("Loading...", "You are not allowed...", "not configured yet" + Open settings), `brainbox_ai.ChatPage` and `brainbox_ai.ChatWidget` (empty containers the SDK fills).

Styles the systray button, hides the SDK's corner button in sidebar mode (the systray icon replaces it), hides the global chat while the AI Assistant app is open, and sizes the page and widget containers. The chat's own look comes from the SDK.

Views, tools and documentation

views/res_config_settings_views.xml

settings screen + action

- `res_config_settings_view_form`: inherits Odoo's settings form (`base.res_config_settings_view_form`, priority 95) and adds a "Brainbox AI" app section, visible to administrators only, with the blocks Connection, Display, Roles and Branding. The key field is a password field.
- `action_brainbox_ai_config_settings`: window action that opens these settings (used by the Configuration menu and the "Open settings" buttons).

views/brainbox_ai_menus.xml

AI Assistant app and menus

- `action_brainbox_ai_chat_page`: client action with tag `brainbox_ai.chat_page`.
- `menu_brainbox_ai_root` "AI Assistant" (home menu, internal users, sequence 45, icon from `static/description/icon.png`) with `menu_brainbox_ai_chat` "Chat" and `menu_brainbox_ai_config` "Configuration" (administrators only).

tools/sync_sdk.ps1

PowerShell helper

Copies `brainbox-web-sdk.js` from a local SDK checkout (`-SdkPath`) or downloads it from `cdn.jsdelivr.net/npm/spres-web@<Version>` (default version 2) into `static/lib/...`, then prints the bundled SDK version. Not used by Odoo at runtime.

README.md static/description/icon.png

docs and icon

Developer documentation of the add-on (install, run, configure, modes, embedding, permissions, proxy, updating the SDK) and the app icon shown in the Odoo home menu and Apps.

9 The web SDK (sdk-web)

sdk-web is one JavaScript file with no dependencies. Any website can show the Brainbox chat by adding two small pieces of code. Odoo uses the very same file.

The plain-HTML snippet

```
<script src="https://port.smartpowerbilling.com/sdk/brainbox-web-sdk.js"></script>
<!-- or from the CDN: https://cdn.jsdelivr.net/npm/spres-web@2 -->
<script>
  const bb = Brainbox.init({
    apiUrl: 'https://port.smartpowerbilling.com',    // the brainBox server
    apiKey: 'pk_live_xxx',                          // a PUBLISHABLE key only
    mode: 'floating', position: 'bottom-right',
    theme: { primary: '#2563EB' },
    branding: { botName: 'Acme Assistant', subtitle: 'Ask us anything' },
    launcher: { text: 'Support chat' },
    quickActions: ['Opening hours?', 'How do I pay my bill?']
  });
</script>
```

Main options

Option	Default	Meaning
apiUrl	"	brainBox address, or a same-site proxy path such as <code>/brainbox_ai/proxy</code> .
apiKey	–	Sent as <code>Authorization: Bearer</code> . Publishable key on public pages; omit it behind a proxy.
tenantId	–	Sent only when set. Optional (the key decides the tenant).
user	{}	<code>{ id, name, email, avatarUrl }</code> : <code>id</code> becomes <code>user_id</code> (per-user history), <code>name</code> is used in greetings.
headers / credentials	{ } / 'same-origin'	Extra headers on every request (e.g. a proxy marker) and cookie policy.
timeout	200000 ms	How long to wait for an answer (an AI answer can take ~3 minutes).
mode / container	'floating'	<code>floating</code> , <code>sidebar</code> , <code>inline</code> or <code>page</code> ; <code>inline</code> and <code>page</code> need a container element.
allowedModes	['floating','sidebar']	Modes the user can switch between with the dock button.
position / offset / zIndex	bottom-right / 24px / 9999	Placement of the floating button and window.

Option	Default	Meaning
width / height	360 / 560	Floating window size (capped to the screen).
sidebarWidth / sidebarTop / sidebarPushContent	380 / 0 / body	Sidebar size, top offset (e.g. a navbar) and the element pushed aside.
defaultOpen	false	Open on first visit; afterwards the user's last state is remembered.
launcher	{type:'button', text:'Chat'}	Button style: <code>button</code> , <code>icon</code> , <code>gif</code> or <code>none</code> (open from your own UI).
theme	primary #b93fff, panel #fbf1ff	Colours, text colour, corner radius, optional font.
branding	botName 'Brainbox AI'	<code>botName</code> , <code>title</code> , <code>subtitle</code> , <code>logoUrl</code> , <code>botAvatarUrl</code> .
welcomeMessages / quickActions / placeholder	greeting / [] / 'Type message...'	Intro bubbles (<code>{{name}}</code> , <code>{{botName}}</code>), suggested questions, input hint.
features	history, upload, emoji, modeSwitch, newChat on; export off	Turn interface features on or off.
context	–	Function returning text added before each question ("Context: ... Question: ..."), e.g. the record being viewed.
storageKey / locale / onEvent	'bb-web' / browser / –	Browser-memory namespace, time format, event callback (open, close, message, response, error, ...).

The SDK also offers methods (`open`, `close`, `send`, `newChat`, `loadSession`, `setMode`, `updateConfig`, `destroy` ...) and a headless `Brainbox.Client` for using the API without any interface. Older v1 code (`BrainboxWebSDK`, `BrainboxWebWidget`) keeps working.

How Odoo's settings become SDK options

Odoo setting / situation	Brainbox.init() option set by brainbox_service.js
(always)	<code>apiUrl: '/brainbox_ai/proxy', headers: {'X-Brainbox-Proxy': '1'}, credentials: 'same-origin'; no apiKey, no tenantId</code>
Logged-in user	<code>user: { id: <Odoo uid>, name }</code> (the id only separates browser memory; the proxy sets the real <code>user_id</code>)
Display mode	Floating → <code>mode: 'floating'</code> ; Sidebar → <code>mode: 'sidebar'</code> (both with <code>allowedModes: ['floating', 'sidebar']</code>); Systray → <code>mode: 'floating', allowedModes: ['floating'], launcher.type: 'none'</code> ; Disabled → no global chat
Position	<code>position</code>
Primary / Panel color	<code>theme.primary / theme.panel</code>
Bot name / Header title / subtitle	<code>branding.botName / branding.title / branding.subtitle</code>
Logo or Logo URL	<code>branding.logoUrl</code> (uploaded logo → <code>/brainbox_ai/logo?v=...</code>)
Launcher text	<code>launcher.text</code>
Welcome message / Quick actions	<code>welcomeMessages / quickActions</code> (one per line)
SDK script URL	Which SDK file is loaded (bundled copy when empty or on failure)
Odoo layout (fixed)	<code>zIndex: 990</code> (below Odoo menus and dialogs), <code>sidebarTop</code> = navbar height, <code>sidebarPushContent: '.o_action_manager'</code> , <code>storageKey: 'bb-odoo:<db>'</code>
AI Assistant app	<code>mode: 'page'</code> , its own container and <code>storageKey</code>
<code><widget name="brainbox_chat"/></code>	<code>mode: 'inline'</code> , <code>context</code> = record description, <code>branding.title</code> = the widget's title

FOR DEVELOPERS: PROXY MODE ON ANY SITE

A website with logged-in users should copy Odoo's approach: a small endpoint on its own domain that checks the session, adds `Authorization: Bearer sk_live_xxx`, the tenant and the user, and forwards to brainBox; then `Brainbox.init({ apiUrl: '/your/proxy', credentials: 'same-origin' })` without a key. When `tenantId` and `user.id` are not set, the SDK does not send them, so the proxy decides.

10 Security and known limitations

What is protected, and how

Risk	Protection
Secret key stolen from a browser	The key is stored only on the Odoo server, never included in the settings sent to the browser, never shown again in the form, and only administrators can change it.
One company reading another's data	The tenant comes from the key; a different tenant in the request is refused (403). Searches, conversations and cached answers are all limited to that tenant.
A user pretending to be someone else or claiming a higher role	The Odoo proxy overwrites user id, name, role and tenant with the real logged-in values. Publishable keys are always "public" on brainBox.
Restricted knowledge leaking through the AI	Documents are filtered by audience before the AI sees them; the cache is separated by role; the AI is told not to reveal other customers' details.
Reading another user's conversations	Conversations are stamped with the user id; other users get "Session not found" (existence is not revealed).
Misuse of Odoo as an open relay	Only logged-in users in Allowed groups; only 7 chat endpoints are forwarded; training, ingestion and key management are blocked (404).
Cross-site request forgery	The proxy requires the custom header <code>X-Brainbox-Proxy: 1</code> , which another website cannot send to Odoo.
Oversized or malicious uploads	10 MB limit (Odoo and brainBox), JSON limited to 512 KB, conversation ids validated; brainBox accepts only JPEG/PNG/GIF/WebP as images and strips paths from file names.
Malicious content in answers or logo	The SDK renders answers with a safe formatter (no raw HTML; only http/https/mailto links). The logo is served with a strict Content-Security-Policy.
Stolen key database	brainBox stores only SHA-256 fingerprints of keys; keys can expire and be revoked; the admin token is compared in constant time.

Known limitations (as of this version)

Limitation	Details
No streaming	The answer arrives all at once; the widget shows "Thinking..." meanwhile. A slow answer can take up to ~3 minutes (the Odoo proxy waits 200 s, then shows a timeout message).
Uploads are stored, not learned from	Files and images attached in the chat are saved on the brainBox server (in a temporary folder) and noted in the conversation, but the AI does not read them when answering and they are not added to the knowledge base. Teaching the AI is done through the training tools with a secret key.
Answer cache keyed on the first 50 characters	Answers are cached for 1 hour per tenant and role using only the first 50 characters of the question. Two different questions that start with the same 50 characters can receive the same answer. This especially affects the embedded form widget, because each question there starts with the same "Context: Odoo record: ..." line for a given record. Worth fixing in brainBox (e.g. hash the whole question).
Test connection does not check the key	<code>/api/health</code> is public, so a successful test proves only that the server is reachable. A wrong key or a mismatching Tenant ID appears at the first real question (401 / 403).
Back-office only	The add-on adds the chat to Odoo's back-office, not to the Odoo website or customer portal pages.
Publishable keys are open by nature	brainBox accepts requests from any website (CORS open) and has no per-key website allow-list or rate limiting yet (listed as next steps). A copied pk_ key can be used elsewhere, but only for public answers. A pk_ key can also list conversations that are not tied to a user.
History depends on brainBox	All conversations live on brainBox. If it is down, Odoo shows an error banner with Retry. Per-user history is tied to <code>odoo:<database>:<user id></code> ; renaming the Odoo database gives users a fresh history.
Per-browser preferences	The open/closed state, docked/floating choice and last conversation are remembered per browser, not per Odoo user account.
Staff dashboard	A web dashboard for staff (keys, knowledge, conversations) is being built and is not covered here; today keys are managed with the CLI or the admin API.

11 Glossary

Term	Meaning in plain words
brainBox	Your AI server application. It stores each company's knowledge, searches it, asks an AI model to write answers and keeps the conversations.
Tenant	One customer company on brainBox: a sealed compartment with its own knowledge, conversations and keys.
SDK	"Software development kit": a ready-made building block that developers drop into their software. Here: the chat window (sdk-web, sdk-react, sdk-react-native).
Add-on / module	A package that extends Odoo. <code>brainbox_ai</code> is the Odoo add-on for Brainbox.
Proxy	A middleman that receives a request and passes it on. Odoo's proxy passes chat requests to brainBox, adding the secret key and the user's identity.
API	The set of addresses (such as <code>/api/chat</code>) through which programs talk to brainBox.
API key	A long password that identifies a client program and its tenant. Publishable (pk_): safe in public pages, public answers only. Secret (sk_): servers only, full access to one tenant.
Admin token	The operator's master password on the brainBox server, used only to create, list and revoke keys.
Role	Who is asking (admin, internal, customer, vendor, public). It decides which knowledge may be used to answer.
Audience	Label on each piece of knowledge saying who it is for (public, customer, vendor, internal, admin).
Session / conversation	One chat thread with its messages, stored on brainBox, owned by one user.
Knowledge base	The company's documents, FAQs, tickets and records that brainBox has learned, cut into passages.
Embedding / vector search	Each passage is turned into a list of numbers that captures its meaning; the question is turned into one too, and the closest passages are found ("semantic search"). Stored in PostgreSQL with pgvector.
AI model (LLM)	The language model that writes the answer from the passages found. brainBox uses Ollama, with OpenAI as a fallback.
Training / ingestion	Adding knowledge to a tenant (files, text, external APIs such as a helpdesk). Requires a secret key; not possible through Odoo's proxy.
Cache	A short-term memory of recent answers (1 hour, per tenant and role) so repeated questions are answered instantly.
System parameter	Odoo's key-value settings store (<code>ir.config_parameter</code>), where the add-on saves its settings.
AbstractModel	An Odoo model that holds code but no database table.
Systray	The row of small icons at the top right of Odoo.
CORS / CSRF	Browser security rules about which websites may call which servers, and a type of attack where another site tricks a browser into sending requests. The proxy design avoids both problems.

12 Where to find everything

What	Where
ON THIS COMPUTER (C:\USERS\USER\DOCUMENTS\PERSONAL\...)	
Odoo add-on	<code>odoo\custom_addons\brainbox_ai\</code>
Odoo source, config, Python	<code>odoo\odoo-bin</code> , <code>odoo\odoo.conf</code> , <code>odoo\.venv\Scripts\python.exe</code>
Web SDK (sdk-web)	<code>ai\sdk-web\</code> (<code>brainbox-web-sdk.js</code> , <code>README.md</code> , <code>widget-example.html</code> , <code>examples\</code>)
React SDK / mobile SDK	<code>ai\sdk-react\</code> / <code>ai\sdk-react-native\</code>
brainBox backend	<code>ai\brainBox\</code> : <code>app\main.py</code> (start-up, routes), <code>app\dependencies.py</code> (key checks), <code>app\apikeys.py</code> (key storage), <code>app\permissions.py</code> (roles/audiences), <code>app\api\chat.py</code> , <code>app\api\upload.py</code> , <code>app\api\admin.py</code> , <code>app\cli.py</code>
brainBox secrets	<code>ai\brainBox\.env</code> on the server (database, admin token, AI settings). Never share or commit it.
This document	<code>ai\docs\odoo-integration-guide.html</code> and <code>ai\docs\Brainbox-Odoo-Integration-Guide.pdf</code>
IN ODOO	
Settings	Settings > Brainbox AI (administrators), or AI Assistant > Configuration
Full-screen chat	Home menu > AI Assistant > Chat
Stored settings	Settings > Technical > System Parameters (developer mode), keys starting with <code>brainbox_ai.</code>
Install / update	Apps > Update Apps List > "Brainbox AI"
ON THE INTERNET	
brainBox server	<code>https://port.smartpowerbilling.com</code> (health: <code>/api/health</code>)
Hosted SDK file	<code>https://port.smartpowerbilling.com/sdk/brainbox-web-sdk.js</code>
SDK on the CDN	<code>https://cdn.jsdelivr.net/npm/spres-web@2</code> (npm package <code>spres-web</code>)
ON THE BRAINBOX SERVER	
Manage keys	<code>python -m app.cli keys create list revoke</code> (or inside Docker: <code>docker exec -i brainbox-api python -m app.cli keys ...</code>)
Admin API	<code>/api/admin/keys</code> with header <code>X-Admin-Token</code>
Logs	<code>brainbox.log</code> (brainBox) and the Odoo server log (proxy warnings)